

A
PROJECT DESIGN REPORT
ON
Ai Cultural Text Simplifier

For the subject **Project Phase I**
Submitted in partial fulfillment of the requirement for the award of
Bachelor of Technology
In
Computer Science and Engineering
WALCHAND INSTITUTE OF TECHNOLOGY
(An Autonomous Institute)
Affiliated to
Punyashlok Ahilyadevi Holkar Solapur University
By

Name	Roll. No.	Exam Seat No.
Vaibhavi Kharawane	16	2202111088
Bhavana Kalpavruksha	17	2202111076
Sakshi Kapse	18	2202111093
Srushti Kapse	19	2202111092

Under Guidance of
Dr. Mrs. R. K. Dixit



DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING
WALCHAND INSTITUTE OF TECHNOLOGY
SOLAPUR - 413006
(2025-26)



CERTIFICATE

This is to certify that the project entitled

Ai cultural text simplifier

is

Submitted by

Name	Roll. No.	Exam Seat No.
Vaibhavi Kharawane	16	2202111088
Bhavana Kalpavruksha	17	2202111076
Sakshi Kapse	18	2202111093
Srushti Kapse	19	2202111092

as a part of Project Design Report for the subject **Project Phase I.**

Dr. Mrs. R. K. Dixit
Project Guide

Dr.Mrs.A.M.Pujar
Dept. of Computer Science and Engineering

(Dr.V.A.Athavale)
Principal

**DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING
WALCHAND INSTITUTE OF TECHNOLOGY
SOLAPUR
(2025-26)**

INDEX

Sr No.	Topic	Page No.
1.	Abstract	4
2.	Introduction	5
3.	Background Study	8
4.	Objectives	10
5.	Technologies Required	11
6.	Proposed Work	15
7.	Uml Diagrams	19
8.	Work Planned For Next Semester	20
9.	Conclusion/Summary	21
10.	References	22

1. Abstract

The **AI Cultural Text Simplifier** project is an innovative initiative aimed at bridging the linguistic and cultural gap between the richness of **ancient Sanskrit literature** and the understanding of **modern readers**. It harnesses the power of **Artificial Intelligence (AI)** to automatically extract, translate, and simplify Sanskrit texts into **clear and contextually accurate Hindi**.

The system integrates multiple advanced AI components in a unified pipeline — **Optical Character Recognition (OCR)** using Tesseract for accurate Sanskrit text extraction from images or manuscripts, the **Google Translate API** for reliable and context-aware Hindi translation, and the **Gemini API** for deep semantic simplification and glossary generation of key Sanskrit terms. Through these technologies, the model ensures **linguistic precision, contextual preservation, and cultural sensitivity**.

Beyond translation, the project emphasizes the importance of **accessibility and learning** by providing simplified explanations and an automatically generated glossary for key cultural and philosophical terms. This makes the tool especially valuable for **students, educators, linguists, researchers, and enthusiasts** interested in Indian heritage and classical studies.

Ultimately, this project demonstrates how AI can be effectively applied to **preserve cultural heritage** while making traditional knowledge **more accessible and understandable** to new generations, thereby promoting both **technological innovation** and **cultural appreciation**.

2. Introduction

Language has always been a powerful medium to convey culture, knowledge, and philosophy. India's ancient heritage, written extensively in **Sanskrit**, encompasses immense wisdom through texts like the *Mahabharata*, *Valmiki Ramayana*, *Bhagavad Gita*, *Upanishads*, and numerous other scriptures. However, Sanskrit is a highly structured and complex language that is no longer commonly used in everyday life. As a result, the modern generation finds it difficult to comprehend or relate to these texts due to linguistic barriers. The problem of **understanding Sanskrit literature** thus becomes not only a linguistic challenge but also a **cultural and educational** one.

This project, titled **"AI-Based Cultural Text Simplifier,"** aims to bridge this gap by leveraging Artificial Intelligence and Natural Language Processing (NLP) techniques. The system takes Sanskrit texts — either typed or extracted from images — and converts them into **simplified Hindi** that is easy for students, educators, and general readers to understand. The goal is to preserve cultural literature while enhancing accessibility for people who do not possess advanced Sanskrit knowledge.

Problem Definition

The primary problem addressed in this project is the **difficulty of understanding classical Sanskrit texts** due to their grammatical complexity and archaic vocabulary. Traditional translation approaches often fail to capture the contextual and cultural essence of the text. Additionally, manual translation and simplification require domain expertise and are time-consuming. Therefore, there is a strong need for an automated system capable of translating and simplifying Sanskrit texts accurately and efficiently.

Approach

To solve this problem, the system integrates **multiple AI-based modules** that work together seamlessly:

1. **Optical Character Recognition (OCR):**
Sanskrit text from scanned documents or images is extracted using the *Tesseract OCR engine*. This module ensures that even printed or handwritten scripts can be processed digitally.
2. **Text Preprocessing:**
The extracted text is cleaned by removing noise, correcting spacing, and normalizing the characters before translation.
3. **Machine Translation (Google Translator API):**
The preprocessed Sanskrit text is sent to the **Google Translator API**, which translates it into standard Hindi. Google's translation service is chosen for its accuracy, multilingual support, and pre-trained neural models.

4. **Text Simplification and Glossary Generation (Gemini API):**

The translated Hindi text is then processed by **Google Gemini 1.5 Flash**, which simplifies it into easy-to-understand Hindi and generates **glossaries** for important Sanskrit terms. Named Entity Recognition (NER) is also applied using the *spaCy* model to identify Sanskrit terms or entities that require definition.

5. **Output Display:**

The final simplified Hindi text and glossary are presented to the user through an interactive web interface built with **Streamlit**.

This approach ensures a smooth, end-to-end automated process from text extraction to final simplification and explanation.

Importance of the Project

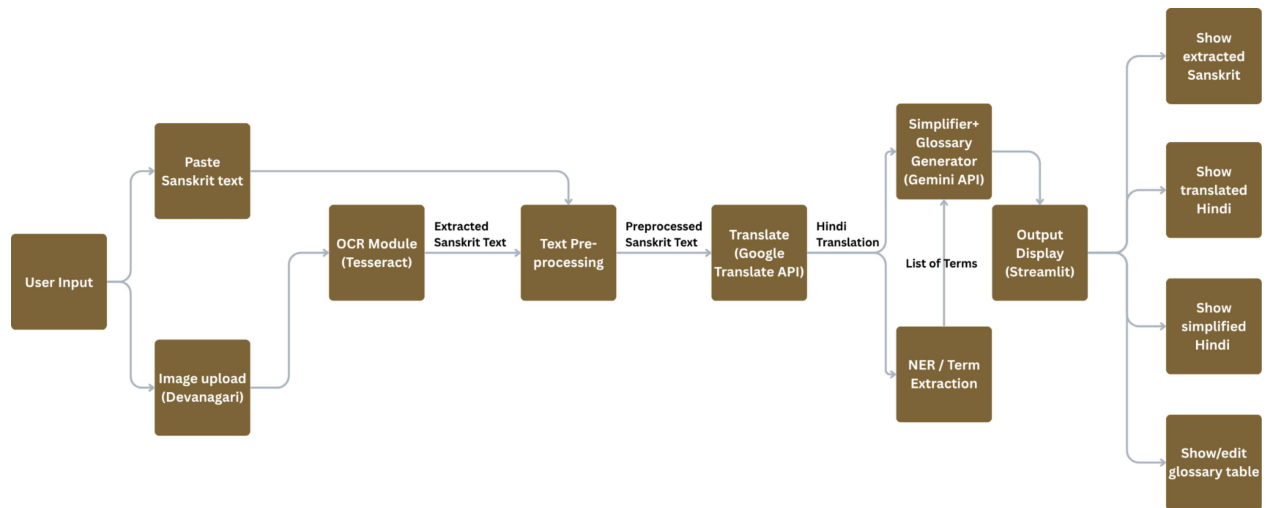
This project holds immense value in the educational and cultural domains. By simplifying Sanskrit literature, it enables easier learning, research, and appreciation of ancient texts. Teachers, students, and enthusiasts can use it as a learning aid, while researchers can analyze cultural concepts without being hindered by language barriers. Moreover, integrating AI-based translation and simplification methods demonstrates the **application of machine learning and NLP in cultural preservation** — a key field in computational linguistics.

Scope of the Project

The project's scope includes:

- OCR for Sanskrit image-to-text conversion
- Machine translation (Sanskrit → Hindi)
- Text simplification using generative AI models
- Automatic glossary generation
- A web-based interface for easy access and display

Block Diagram & Working



Working Explanation:

1. **User Input:** User uploads a Sanskrit image or enters text.
2. **OCR Module:** Tesseract OCR extracts text from the image.
3. **Text Preprocessing:** Cleaning and normalization of Sanskrit text.
4. **Google Translator API:** Converts Sanskrit into Hindi text.
5. **Gemini Simplifier + Glossary:** Simplifies Hindi translation and generates glossary using AI.
6. **Output Display:** Simplified Hindi and glossary are shown in a user-friendly interface.

3. Background Study

Understanding and preserving Sanskrit literature is an important part of India's cultural and linguistic heritage. However, due to the language's grammatical complexity and limited modern usage, most readers find it difficult to comprehend Sanskrit texts such as the *Mahabharata*, *Ramayana*, or *Bhagavad Gita*. To bridge this gap, researchers and technologists have developed several Natural Language Processing (NLP) systems for translation, simplification, and linguistic analysis of Sanskrit texts.

Existing Systems

1. Sanskrit OCR and Digitization Systems

Projects such as **Google OCR for Devanagari** (Smith, 2007), **SanskritOCR**, and the **Digital Corpus of Sanskrit (DCS)** (Hellwig, 2010) have contributed to the digitization of ancient manuscripts. These systems use Tesseract or similar OCR engines to recognize Devanagari script, though accuracy depends heavily on image quality and font clarity.

2. Machine Translation Systems

Several machine translation (MT) models exist to translate Sanskrit into Hindi or English.

- **Google Translate** supports Sanskrit ('sa') and provides direct translation to Hindi ('hi') using neural MT models based on the Transformer architecture (Vaswani et al., 2017).
- **AI4Bharat's IndicTrans2** (Ramesh et al., 2023) offers a multilingual neural machine translation framework for Indian languages, including Sanskrit and Hindi.
- **Anusaaraka System** (Bharati et al., 1995) developed by IIT Hyderabad and IIIT Hyderabad provided early Sanskrit-to-Hindi translation using rule-based methods but with limited linguistic scope.

3. Text Simplification Systems

Traditional NLP simplification systems like **SIMPAC** (Aluísio et al., 2008), **Newsela Simplifier**, and **OpenAI GPT-based** systems focus on reducing linguistic complexity in modern languages such as English. However, similar systems for Indian languages, especially Sanskrit-to-Hindi, remain underdeveloped, and existing systems prioritize translation accuracy over readability.

4. Named Entity Recognition (NER) and Glossary Generation

Research in Sanskrit NER and word sense disambiguation, such as **Amrita Sanskrit WordNet** (Kulkarni et al., 2010) and the **Sanskrit Heritage Platform** (Gérard, 2012), helps identify proper nouns and specialized terms. However, these systems are largely academic and lack simplified explanations suitable for general readers.

Gap in Existing Systems

Despite advances in OCR and translation, there is **no unified system** that combines:

- Sanskrit text extraction from both image and text sources,
- Preprocessing and cleaning of OCR noise,
- Accurate Sanskrit-to-Hindi translation,
- Simplified Hindi generation for easy comprehension, and
- Glossary creation explaining Sanskrit terms in simple Hindi.

Most prior works are either rule-based or academic in focus, lacking a cohesive pipeline that integrates OCR, translation, simplification, and glossary generation.

Proposed Approach

This project proposes an **AI-based Cultural Text Simplifier** that integrates:

- **Tesseract OCR** for Sanskrit text extraction,
- **Google Translate API** for Sanskrit-to-Hindi translation, and
- **Google Gemini 1.5 Flash** for text simplification and glossary generation using **NER (spaCy)** for term identification.

By combining these state-of-the-art pre-trained models, the system aims to make Sanskrit cultural and literary texts accessible to students, educators, and general readers, preserving contextual accuracy while improving readability and cultural comprehension.

4. Objectives

The primary objective of this project is to design and develop an **AI-based Cultural Text Simplifier** that simplifies Sanskrit texts into easily understandable Hindi while preserving their cultural and contextual meaning. This system aims to make ancient Indian literature more accessible to students, educators, and the general public.

The specific objectives of the project are as follows:

1. **To extract Sanskrit text** from manuscripts or scanned documents using Optical Character Recognition (OCR) techniques.
 - Utilize *Tesseract OCR* to accurately detect and extract Devanagari script from image-based sources.
2. **To perform Sanskrit-to-Hindi translation** using a reliable and pre-trained translation model.
 - Integrate the *Google Translate API* to translate Sanskrit text into grammatically correct and contextually accurate Hindi.
3. **To simplify the translated Hindi text** for better comprehension by students and general readers.
 - Employ the *Google Gemini 1.5 Flash/2.5 API* to generate simplified, reader-friendly Hindi text while maintaining the essence of the original Sanskrit meaning.
4. **To generate a glossary of Sanskrit terms** for deeper understanding of cultural and religious contexts.
 - Use *Named Entity Recognition (NER)* through *spaCy* to identify key Sanskrit terms and provide simple Hindi explanations for each term.
5. **To create a user-friendly interface** that allows users to input Sanskrit text (typed or scanned) and view:
 - The original Sanskrit text,
 - The translated Hindi version,
 - The simplified Hindi output, and
 - A glossary of key terms and their meanings.
6. **To ensure readability and cultural relevance** of the simplified output by maintaining linguistic accuracy and semantic integrity.
7. **To evaluate system performance** in terms of translation accuracy, simplification quality, and glossary precision through user testing and comparative analysis.

5. Technologies Required

The development of the **AI-Based Sanskrit Text Simplifier** involves the integration of multiple technologies from the domains of Artificial Intelligence (AI), Natural Language Processing (NLP), and Web Development. Each component plays a crucial role in ensuring accurate OCR extraction, reliable translation, efficient simplification, and an interactive user experience. The following section describes the key technologies used in this project.

1. Python

Python is the primary programming language used in this project due to its simplicity, readability, and the wide availability of libraries for AI and text processing. It serves as the backbone of the backend system, enabling integration of OCR, translation, and simplification modules.

- **Why Python:**

- Easy integration with APIs like Google Translate and Gemini.
- Extensive NLP and AI libraries such as `spaCy`, `pytesseract`, and `transformers`.
- Strong community support and cross-platform compatibility.

2. FastAPI

FastAPI is used for developing the **backend RESTful API** of the project. It provides high performance, automatic data validation, and easy asynchronous processing.

- **Purpose in the project:**

- Acts as a bridge between the frontend and various AI modules.
- Exposes endpoints such as `/api/ocr`, `/api/translate`, and `/api/simplify`.
- Ensures smooth communication between components and handles user requests efficiently.

- **Key Advantages:**

- High speed comparable to Node.js.
- Built-in documentation using Swagger UI.
- Easy integration with machine learning and AI models.

3. Streamlit

Streamlit is used for creating the **frontend interface** of the system. It allows quick development of interactive web applications directly in Python.

- **Role in the project:**
 - Enables users to upload images or enter text manually.
 - Displays OCR results, translated Hindi text, simplified output, and glossary.
- **Benefits:**
 - Fast and lightweight for prototyping AI-based tools.
 - No need for complex HTML, CSS, or JavaScript code.
 - Real-time updates with minimal latency.

4. Tesseract OCR

Tesseract is an open-source Optical Character Recognition (OCR) engine developed by Google. It is used to extract Sanskrit text from image inputs.

- **Usage in project:**
 - Reads Devanagari script using the trained data file (`san.traineddata`).
 - Converts scanned Sanskrit text into machine-readable format for further processing.
- **Advantages:**
 - High accuracy for Indian scripts.
 - Open-source and easily configurable.
 - Supports multiple languages including Sanskrit, Hindi, and English.

5. Google Translate API

The **Google Translate API** is used for the **Sanskrit-to-Hindi translation** stage. It leverages Google's multilingual neural machine translation (NMT) model.

- **Purpose in project:**

- Automatically converts extracted Sanskrit text into fluent Hindi.
- Reduces the need for building and training a custom translation model.
- **Advantages:**
 - Pre-trained on large multilingual datasets.
 - Provides consistent and accurate translations.
 - Easy to integrate with Python using the [googletrans](#) or official [google-cloud-translate](#) library.

6. Google Gemini API

The **Gemini 1.5 Flash/2.5 API** is a large multimodal generative AI model developed by Google DeepMind. In this project, it is used for **Hindi text simplification and glossary generation**.

- **Usage:**
 - Simplifies translated Hindi text to make it easy for students and general readers.
 - Generates a glossary explaining key Sanskrit words and their Hindi meanings.
- **Advantages:**
 - Context-aware understanding and rewriting capability.
 - Multilingual support and cultural sensitivity.
 - Faster inference time and cost-efficient for text-only use cases.

7. spaCy

[spaCy](#) is an open-source NLP library used for **Named Entity Recognition (NER)** in this project. It identifies key Sanskrit terms and entities that require glossary explanations.

- **Purpose:**
 - Detects proper nouns, deities, locations, or important terms in Sanskrit text.
 - Supplies the Gemini model with terms for generating meaningful glossaries.
- **Advantages:**

- Efficient and lightweight NLP engine.
- Multilingual support using models like `xx_ent_wiki_sm`.
- Easy to integrate with FastAPI and Python-based AI workflows.

8. JSON Data Exchange

The backend and frontend communicate using **JSON (JavaScript Object Notation)** format.

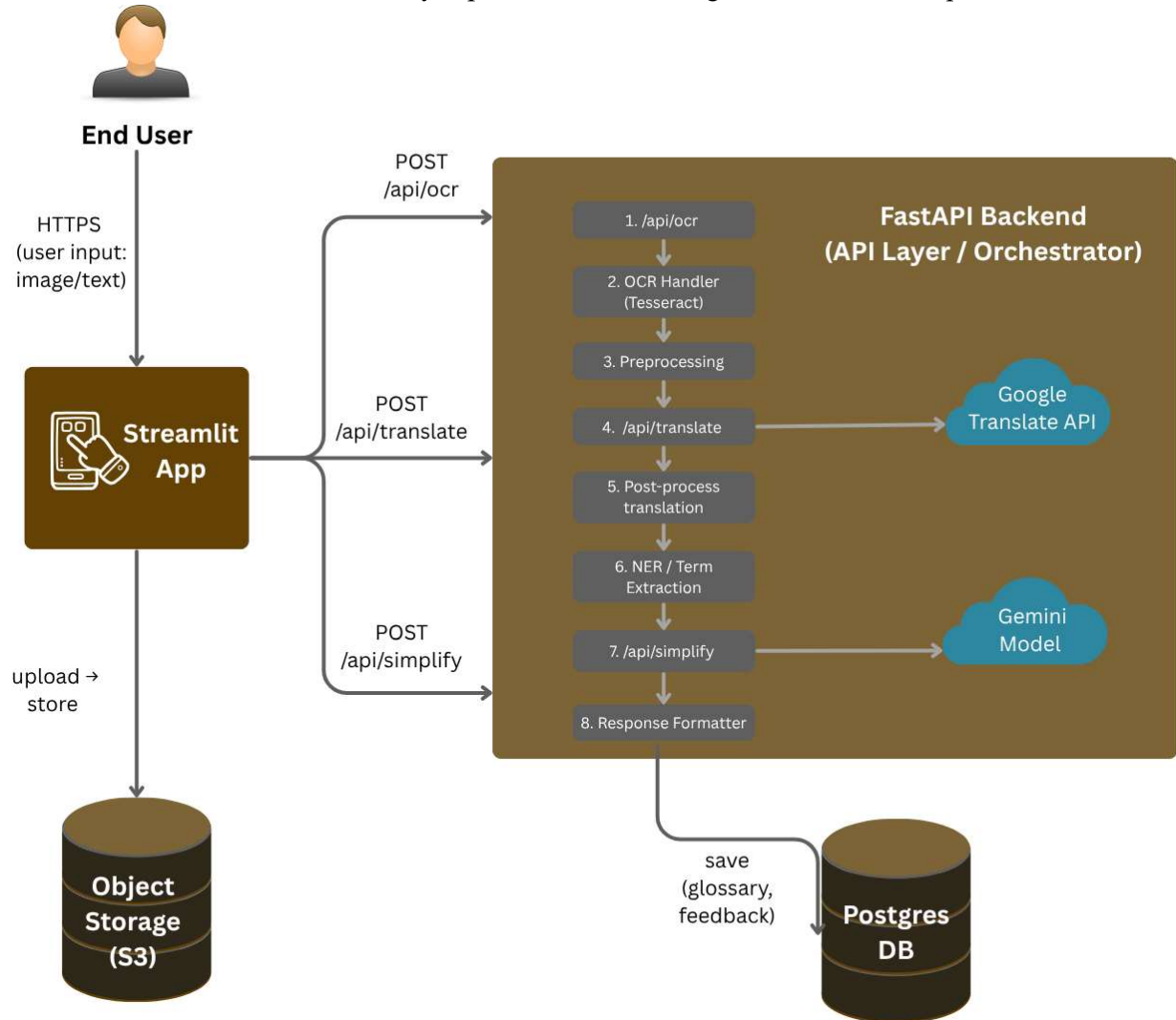
- **Role:**
 - Used for transferring OCR output, translated text, simplified Hindi, and glossary lists.
 - Ensures a consistent, lightweight, and structured format for data exchange.

9. Development Environment

- **Operating System:** Windows 10 / Linux
- **IDE / Tools:** Visual Studio Code, Anaconda
- **Version Control:** Git and GitHub
- **Testing Tool:** Streamlit interface and REST API testing via Postman.

6. Proposed Work

The system implements an end-to-end pipeline that converts Sanskrit input (image or raw text) into simplified Hindi with a glossary of Sanskrit terms. It follows a client-server architecture: a Streamlit-based frontend for user interaction and a FastAPI backend that orchestrates OCR, text preprocessing, translation, term extraction, and simplification using external AI services. The architecture is modular to allow easy replacement or scaling of individual components.



Major Components

1. User Interface (Streamlit)

- Collects input: image upload (Devanagari script) or pasted Sanskrit text.
- Displays pipeline outputs: extracted Sanskrit, raw Hindi translation, simplified Hindi, and glossary table.

- Communicates with the backend via HTTPS **POST** endpoints.

2. FastAPI Backend (Orchestrator)

- Exposes REST endpoints (**/api/ocr**, **/api/translate**, **/api/simplify**) and validates requests.
- Orchestrates internal pipeline steps and marshals data between modules and external APIs.
- Handles caching, persistence, error handling, and logging.

3. OCR Module (Tesseract)

- Runs locally on the backend host. Uses Devanagari trained data (e.g., **san.traineddata** or **hin**) to extract Unicode Sanskrit text from images.
- Returns raw extracted text to preprocessing.

4. Text Preprocessing

- Normalizes Unicode, removes OCR noise, corrects spacing and punctuation, and optionally performs sandhi splitting or lemmatization to improve translation and term matching.

5. Translation Service (Google Translate API)

- Converts preprocessed Sanskrit into Hindi. Large texts are chunked (Google Translate limit ~5k characters/request) and reassembled.
- Backend holds API keys and performs server-side calls.

6. NER / Term Extraction (spaCy or fallback)

- Identifies important Sanskrit tokens (entities, proper nouns, technical terms) that need glossary entries. If spaCy NER is unavailable, a Devanagari-token extractor fallback is used.

7. Simplifier & Glossary Generator (Gemini GenAI)

- Receives the original Sanskrit (or preprocessed), the cleaned Hindi translation, and the extracted term list.
- Produces simplified Hindi text and a glossary of Sanskrit terms with simple Hindi explanations (structured JSON).

8. Storage & Cache

- **Object Storage (S3 or equivalent):** stores uploaded images and processed artifacts.
- **Relational DB (Postgres):** stores glossary entries, user feedback, and audit logs.

Data Flow (Sequence)

1. User uploads image or pastes text in Streamlit.
2. Frontend sends image/text to FastAPI `/api/ocr` or directly to `/api/translate`.
3. Backend runs OCR (if image) → Preprocessing → chunks to Google Translate → receives `raw_hindi`.
4. Post-process translation → run NER to extract `list_of_terms`.
5. Backend builds a prompt combining original Sanskrit, cleaned Hindi, and term list and calls Gemini (`models/gemini-2.5-flash`) to **simplify** and **generate a glossary**.
6. Response is validated, optionally stored (DB) and returned to the frontend for display.

APIs & Protocols

- Frontend ↔ Backend: HTTP(S) REST (`POST /api/ocr`, `POST /api/translate`, `POST /api/simplify`).
- Backend ↔ External Services: HTTPS with API keys (Google Translate; Gemini/GenAI). All external keys are used exclusively server-side.

Security Considerations

- **API keys** are never exposed to the client; they are stored in environment variables or a secret manager.
- **Input validation** and size limits prevent resource exhaustion and abuse.
- **Rate limiting** and request-size checks protect against excessive API costs.
- If deployed publicly, enforce HTTPS, CORS restrictions for known frontends, and authentication for admin features.

Deployment & Scaling

- **Development:** Run Streamlit and FastAPI locally (ports 8501 and 8000).
- **Production:** Containerize components (Docker); orchestrate with Docker Compose or Kubernetes. Use managed services for translation and GenAI to simplify scaling. Horizontal scale the FastAPI app behind a load balancer; use worker queues (Celery/RabbitMQ) for long-running tasks and to avoid blocking HTTP responses.

Reliability & Performance

- Use **caching** for repeated requests (Redis) to lower latency and API costs.
- Offload heavy or slow tasks (large OCR on big images, long simplifications) to asynchronous workers and provide progress feedback or notifications.
- Maintain robust **logging and monitoring** for tracing errors (OCR failures, API timeouts, model errors).

7. UML diagrams

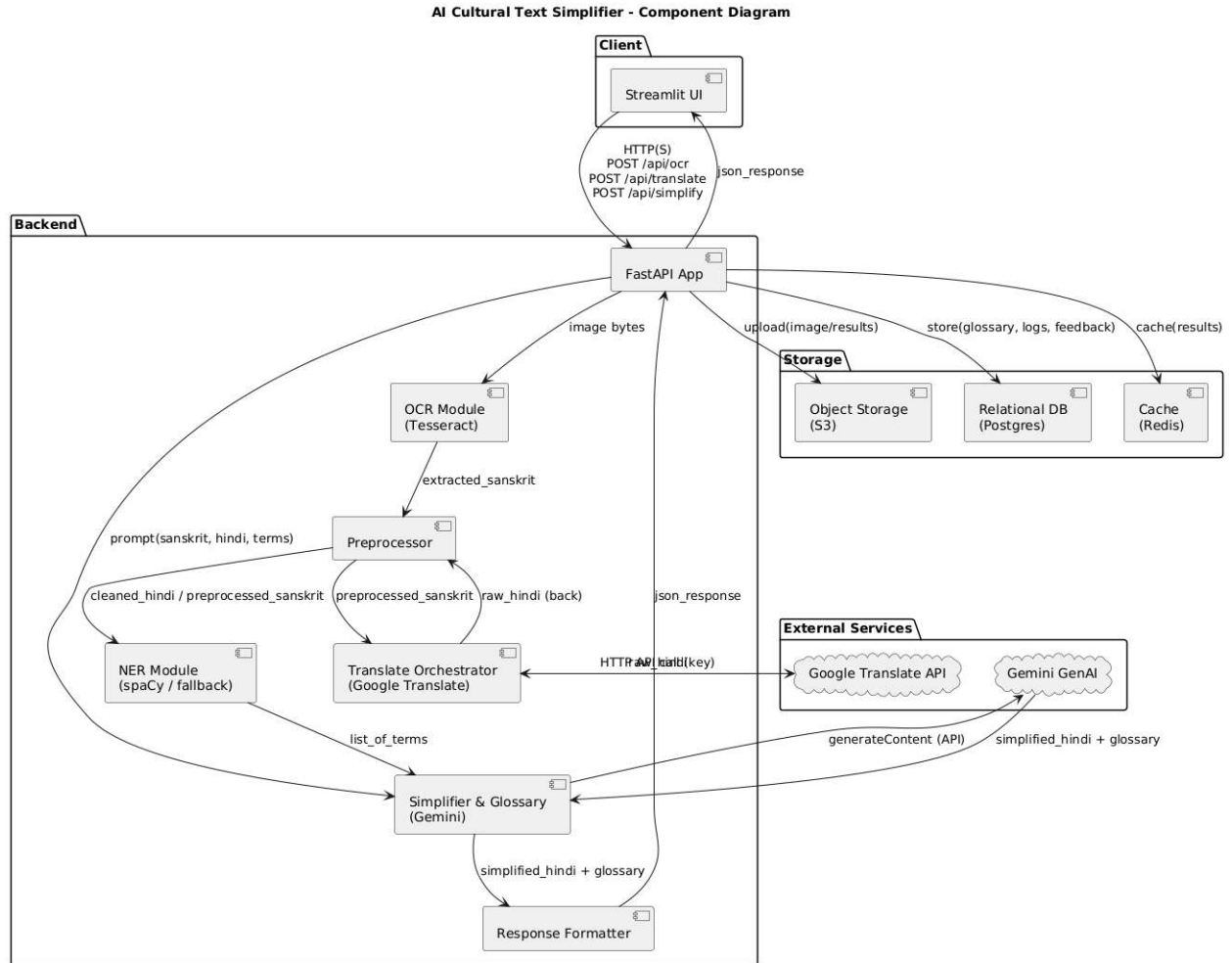


Fig. Component Diagram

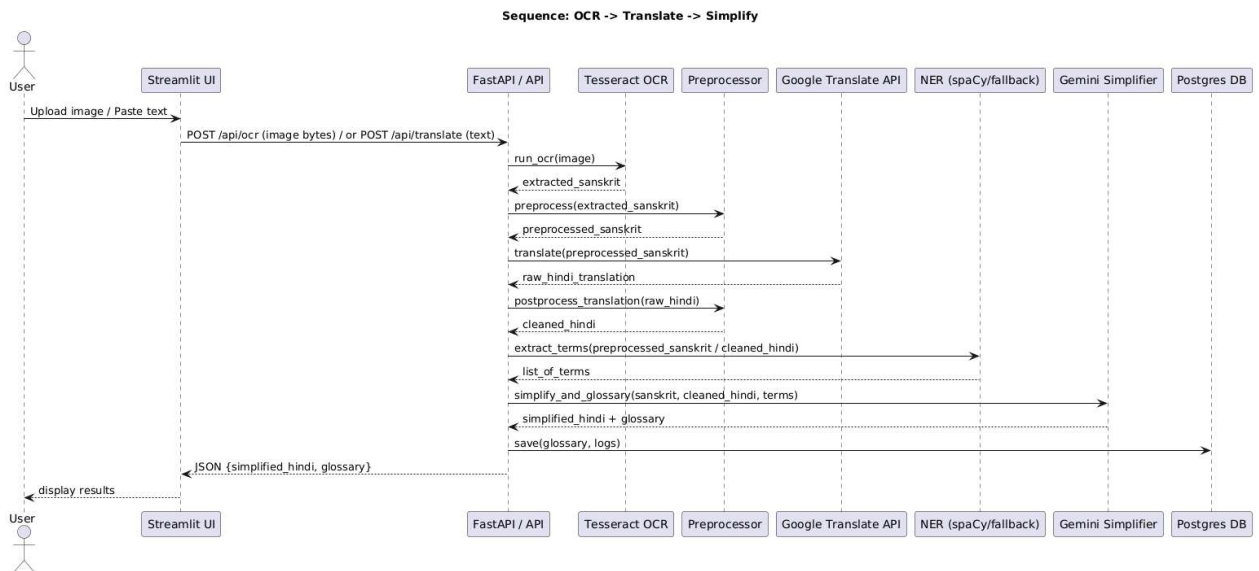


Fig. Sequence Diagram

8. Work Planned for Next Semester

The current phase of the project successfully implements Optical Character Recognition (OCR) for Sanskrit text, translation to Hindi using Google Translate API, and text simplification with glossary generation using Google's Gemini API. However, several important developments are planned for the next semester to enhance the system's usability, functionality, and overall performance.

- 1. Frontend Development:**

A fully functional and user-friendly web interface will be developed. This interface will include user authentication (sign-in and sign-up), file and text input options, and a dashboard to display translation and simplification history.

- 2. Multilingual Support:**

The system can be extended to support multiple Indian languages such as Marathi, Tamil, Bengali, and English. This would make the platform useful for a wider audience and increase accessibility for students and researchers across different linguistic backgrounds.

- 3. Voice Integration:**

Text-to-speech and speech-to-text features can be added to enhance accessibility, allowing users to listen to the simplified Hindi text or input Sanskrit text through voice commands.

- 4. Data Storage and Caching:**

Integration with a database system (such as MongoDB or PostgreSQL) will be implemented to store user data, OCR results, and simplified text outputs. Caching mechanisms will also be introduced to reduce latency and improve response times for frequently accessed data.

- 5. Cloud Deployment:**

The complete backend and frontend will be deployed on a cloud platform such as AWS, Azure, or Google Cloud. This will ensure scalability, continuous availability, and remote accessibility of the system.

- 6. Performance and Accuracy Optimization:**

Efforts will be made to enhance OCR precision for complex Sanskrit scripts and improve translation and simplification efficiency. Preprocessing algorithms will be refined, and prompt engineering for the Gemini API will be optimized.

- 7. Glossary Enrichment:**

The glossary feature will be expanded by integrating additional linguistic resources and improving the Named Entity Recognition (NER) model to identify and define Sanskrit terms more effectively.

- 8. Testing and Evaluation:**

Comprehensive testing will be carried out to evaluate system performance, translation accuracy, simplification clarity, and overall user experience. Feedback from users will be collected to guide further improvements.

9. Conclusion/Summary

The **AI Cultural Text Simplifier** project successfully demonstrates how modern artificial intelligence techniques can bridge the gap between ancient cultural literature and contemporary understanding. By integrating **Optical Character Recognition (OCR)** for Sanskrit text extraction, **Google Translate API** for Sanskrit-to-Hindi translation, and **Google Gemini API** for Hindi simplification and glossary generation, the system creates a unified pipeline that transforms complex traditional texts into simplified and easily understandable language.

This work showcases the potential of combining **Natural Language Processing (NLP)**, **Machine Translation**, and **Generative AI** to preserve and promote India's cultural and literary heritage in a form accessible to students, researchers, and the general public. The modular architecture of the system allows for future enhancements such as user authentication, data storage, multilingual capabilities, and voice integration for better accessibility.

In conclusion, this project not only contributes to the digital preservation and interpretation of Sanskrit texts but also highlights the effective use of AI-driven translation and simplification techniques for promoting cultural education and cross-linguistic understanding.

10. References

1. Google Cloud. (2024). *Cloud Translation API documentation*. Retrieved from <https://cloud.google.com/translate/docs>
2. Google AI. (2024). *Gemini API documentation*. Retrieved from <https://ai.google.dev/docs>
3. Streamlit. (2024). *Streamlit Documentation*. Retrieved from <https://docs.streamlit.io>
4. FastAPI. (2024). *FastAPI Documentation*. Retrieved from <https://fastapi.tiangolo.com>
5. Python Software Foundation. (2024). *Python Language Reference, version 3.11*. Retrieved from <https://docs.python.org/3/>
6. Hugging Face. (2024). *mT5: Multilingual Text-to-Text Transfer Transformer*. Retrieved from https://huggingface.co/transformers/model_doc/mt5.html
7. Google Developers. (2024). *Google Cloud Vision API for OCR*. Retrieved from <https://cloud.google.com/vision/docs>
8. Scikit-learn Developers. (2024). *Natural Language Processing in Python*. Retrieved from https://scikit-learn.org/stable/modules/feature_extraction.html#text-feature-extraction
9. OpenAI. (2023). *Applications of Generative AI in Text Simplification*. Retrieved from <https://openai.com/research>
10. ResearchGate. (2022). *Text Simplification for Low-Resource Languages: A Review of Techniques and Challenges*. Retrieved from <https://www.researchgate.net/publication/366158244>
11. Pandas Development Team. (2024). *Pandas Documentation*. Retrieved from <https://pandas.pydata.org/docs/>
12. NLTK Project. (2024). *Natural Language Toolkit Documentation*. Retrieved from <https://www.nltk.org/>